

Knowledge Graph Report

Knowledge Graph Report

Introduction

This project develops a knowledge graph for London public transportation with a particular focus on the Transport for London (TfL) network, Oyster/contactless fare semantics, accessibility, operators, night services, and journey-planning facts. The domain choice aligns directly with the coursework brief's recommended "public transportation" theme and supports a mixture of operational, accessibility, and fare-related competency questions.

The repository implements a hybrid construction workflow. A manually curated ontology and seed ABox are stored in `ontologies/manual/tfl_kamyar_final.ttl`, the structured pipeline is implemented in `data_retrieval/structured_retrieval/tfl_api_processor.py`, the unstructured pipeline is implemented across `data_retrieval/unstructured_retrieval/`, and the merge step is orchestrated by `main.py`. The final merged artifact in the repository is `ontologies/pipeline_output/final_london_transport_kg.ttl`, which currently contains 17,539 triples and occupies roughly 656 KB on disk.

The public repository linked to this submission is:

- [sadiqkhanfanclub](#)

The important outcome is that the graph covers the coursework's required question set while also exposing where the current build is still incomplete. The current repository snapshot is strong on documentation of intent, source manifests, and competency-question coverage, but it is also a hybrid snapshot rather than a fully clean rebuild from raw inputs. In particular, the manually curated ontology currently provides most of the queryable `http://example.org/tfl#` facts used by the coursework SPARQL queries, while the automated outputs require a namespace-alignment fix before they can contribute seamlessly to the same query space.

Data Source Selection

Structured sources

The primary structured source is the official TfL Unified API, cached locally in `downloads/tfl_api_cache.json`. The committed cache is approximately 79 MB and contains 695 line records across bus, Tube, Overground, DLR, and Elizabeth line services, with 7,949 nested stop-point entries. This choice is appropriate for the coursework because it offers authoritative operational identifiers, line metadata, route membership, and station/stop attributes in a machine-readable form.

The repository also includes a generated source manifest in `docs/generated/tfl-sources.json` and `docs/generated/tfl-sources.md`. The JSON manifest records 1,299 retrievals across the following categories:

- `core`: 1
- `textual`: 2

- `route_sequences`: 695
- `line_stop_points`: 19
- `line_statuses`: 19
- `search`: 3
- `stop_points`: 560

This is valuable from a reproducibility point of view because it shows the intended breadth of the structured collection process, not just the small subset ultimately materialized into the main queryable namespace.

As a secondary structured reference, the repository includes `downloads/naptan_london.csv` (about 4.3 MB). NaPTAN is useful as a normalization and identifier-support resource for stops and stations, particularly where the TfL API returns multiple stop-point variants or where station naming ambiguity arises.

Textual sources

The unstructured pipeline begins from a curated list of 21 Wikipedia articles defined in `data_retrieval/unstructured_retrieval/wiki_scraper.py`. These articles cover ticketing, accessibility, night services, network context, and general transport semantics, including entries such as "Oyster card", "London fare zones", "Step-free access", "Night Tube", "Night buses in London", and "Interchange station".

The source manifest additionally records two non-Wikipedia textual resources that are closer to official evidence:

- TfL step-free guide PDF
- TfL Freedom of Information page for London Overground operator information

This hybrid source strategy is sensible for the domain. Official structured APIs are the best fit for line, stop, and status data, while textual sources are useful for extracting explanatory semantics such as accessibility, concessions, and service concepts that do not always appear in a clean schema from the API.

Source-selection rationale

The data-source selection balances four considerations:

1. **Authority.** TfL API responses and official TfL pages are the most defensible evidence for operational transport facts.
2. **Coverage.** Wikipedia articles broaden the semantic range to include accessibility, fare-policy, and service concepts.
3. **Reproducibility.** Cached JSON, a manifest of retrieval URLs, and committed Turtle outputs reduce dependence on live services during marking.
4. **Coursework fit.** The combination satisfies the requirement to use at least one structured source and one textual source.

Source-related caveats

Two limitations should be acknowledged explicitly.

First, the repository currently preserves the top-level API cache and the generated source manifest, but not the full `downloads/raw/...` snapshot tree referenced by the manifest. Second, the unstructured pipeline refers to raw Wikipedia text snapshots under `data/raw/...`, but those raw text files are not committed in the current repository snapshot. The graph outputs and LLM cache are present, but the full end-to-end provenance trail is therefore incomplete.

Extension Of Existing Ontologies

The core ontology in `ontologies/manual/tfl_kamayar_final.ttl` imports and extends two existing ontologies:

- GTFS (<http://vocab.gtfs.org/terms#>)
- Schema.org (<http://schema.org/>)

This satisfies the coursework requirement to reuse existing ontologies and extend them with domain-specific classes and properties. The main benefits are interoperability and avoiding the unnecessary reinvention of general transport and web-data concepts.

Class extensions

The ontology introduces several subclasses rooted in GTFS and Schema.org concepts. Important examples include:

- `tfl:TfLLine rdfs:subClassOf gtfs:Route`
- `tfl:TfLStation rdfs:subClassOf gtfs:Station`
- `tfl:OysterFareZone rdfs:subClassOf gtfs:Zone`
- `tfl:TfLOperator rdfs:subClassOf gtfs:Agency`
- `tfl:OysterFare rdfs:subClassOf schema:PriceSpecification`
- `tfl:Journey rdfs:subClassOf schema:Trip`

These are then specialized further into subclasses such as `UndergroundLine`, `NightTubeLine`, `BusRoute`, `InterchangeStation`, `ElizabethLineStation`, `PublicToilet`, and `CarPark`. This layered design is appropriate because it keeps the ontology recognizably transport-oriented while allowing the graph to answer the coursework's more domain-specific questions.

Property extensions

The ontology also defines useful transport-specific properties and aligns them upward to GTFS and Schema.org where appropriate. Key examples include:

- `tfl:servedByLine rdfs:subPropertyOf gtfs:route`
- `tfl:operatesInZone rdfs:subPropertyOf gtfs:zone`
- `tfl:routeNumber rdfs:subPropertyOf gtfs:shortName`
- `tfl:lineColour rdfs:subPropertyOf gtfs:color`
- `tfl:operatedBy rdfs:subPropertyOf schema:provider`
- `tfl:journeyOrigin rdfs:subPropertyOf schema:departureStation`
- `tfl:journeyDestination rdfs:subPropertyOf schema:arrivalStation`
- `tfl:fareAmount rdfs:subPropertyOf schema:price`

These choices are technically strong because they allow local modeling decisions to remain compatible with broader vocabularies. They also support clearer reporting and potential reuse outside the coursework.

Domain-specific concepts

The ontology adds concepts that are important for this domain but not cleanly provided by the reused ontologies, including:

- fare concessions
- night services
- station facilities
- accessibility features

- engineering closures
- alternative services
- multi-leg journeys

This is a good example of extending rather than replacing external ontologies: GTFS and Schema.org provide a skeleton, while the TfL ontology adds the domain semantics needed by the competency questions.

Mappings

Structured mapping pipeline

The structured pipeline is implemented in `data_retrieval/structured_retrieval/tfl_api_processor.py`. Its workflow is:

1. Load a local JSON cache if available, otherwise fetch live TfL API data.
2. Iterate through each line and attach stop-point data.
3. Map each line to a domain class according to `modeName`.
4. Create station resources from stop points.
5. Interpret `additionalProperties` as either station facilities or accessibility features.
6. Serialize the result to `ontologies/pipeline_output/structured_london_transport.ttl`.

The mode-to-class mapping is straightforward and domain-appropriate:

- `tube` -> `UndergroundLine`
- `dlr` -> `DLRLine`
- `elizabeth-line` -> `ElizabethLineService`
- `overground` -> `OvergroundLine`
- `bus` -> `BusRoute`

At station level, the pipeline maps stop-point metadata into ontology facts such as:

- `servedByLine`
- `hasFacility`
- `hasAccessibilityFeature`
- `hasStepFreeStreetToPlatform`

This is a reasonable mapping design because it translates source-specific JSON fields into domain semantics that directly support the competency questions.

Unstructured mapping pipeline

The unstructured workflow is distributed across three scripts:

- `wiki_scraper.py`
- `extract_triples.py`
- `triples_to_rdf.py`

The process is:

1. Download text from selected Wikipedia pages.
2. Split text into chunks.
3. Run spaCy named-entity recognition to identify relevant candidate entities.
4. Use an LLM prompt, or cached LLM output, to extract candidate triples.
5. Filter the extracted triples against recognized entities.
6. Map free-text predicates to ontology properties.
7. Serialize the resulting RDF and attach provenance links.

The repository currently ships with `USE_LLM = False`, meaning the build depends on cached triple extraction results stored in `data/caches/triples_cache.json`. That cache currently contains 563 entries. This is useful for reproducibility and offline marking, even though it means the present build is not using a live LLM call at runtime.

The predicate-mapping stage in `triples_to_rdf.py` is particularly important. It normalizes a wide range of surface predicates such as `operatedBy`, `hasStop`, `acceptedOn`, `hasAccessibilityFeature`, and `openedDate` into ontology properties under the TfL namespace. The script also adds `dcterms:source` and `prov:wasGeneratedBy`, which gives the unstructured portion of the graph better provenance than many student projects achieve.

Merge step

`main.py` loads the manual ontology, loads the GTFS ontology, runs the unstructured pipeline, runs the structured pipeline, and serializes the merged graph to `ontologies/pipeline_output/final_london_`

In design terms, this is a good architecture for the coursework because it separates:

- ontology definition
- structured extraction
- unstructured extraction
- final graph assembly

Important integration limitation

The current repository snapshot also exposes the main technical weakness of the build. The automated outputs do not currently share the same namespace as the manual ontology.

- The structured output serializes resources under a Linux file-based `file:///.../ontologies/pipeline_` namespace
- The unstructured output serializes resources under a Windows file-based `file:///.../ontologies/pipeline_` namespace
- The manual ontology and all coursework SPARQL queries use `http://example.org/tfl#`

This means the repository currently contains three parallel TfL namespaces rather than one canonical graph namespace. In practice, this is why the coursework SPARQL queries are answered primarily by the manually curated `http://example.org/tfl#` ABox rather than by the automated outputs. The automated pipeline still adds a large amount of data, but it is not yet fully integrated into the namespace used by the competency-question layer.

This limitation should be documented rather than hidden, because it is the single clearest remaining blocker to claiming a fully automated end-to-end KG population workflow.

Queries

The repository includes 20 competency questions and their SPARQL implementations in `competency_questions/competency_question.txt`. The executable validation harness is `competency_questions/competency_question_test.py`, which loads `ontologies/pipeline_output/final_london_transport_kg.ttl` and runs all 20 questions.

Running that validation script against the current repository snapshot returns non-empty results for all 20 questions, so the recorded CQ answerability score is currently:

- 20/20 answered
- 100% non-empty query coverage

Representative results from the current graph include:

- King's Cross St Pancras is linked to Circle, Hammersmith & City, Metropolitan, Northern, Piccadilly, and Victoria lines.
- The Victoria line terminates at Brixton and Walthamstow Central.
- Stratford is modeled as an interchange served by Central, Jubilee, DLR, Elizabeth line, and Overground services.
- Baker Street has a public toilet facility.
- The Night Tube lines returned by the graph are Central, Jubilee, Northern, Piccadilly, and Victoria.
- Three fare concessions for disabled bus passengers are represented and queryable.

It is also worth noting that the repository contains two slightly different CQ descriptions: one in `README.md` and one in `competency_questions/competency_question.txt`. For the purposes of implementation and validation, the authoritative set is the latter, because it is the one paired with executable SPARQL and a working test script.

Evaluation Methodology

Quality metrics

The most direct quality metric for this coursework is competency-question answerability. On the current repository snapshot, all 20 stored SPARQL queries return at least one answer.

Additional useful quality indicators from the current graph are:

- final merged graph size: 17,539 triples
- queryable `http://example.org/tfl#` station-like instances: 30
- queryable line-like instances under the core ontology hierarchy: 39
- queryable facility-like instances: 7
- queryable accessibility-feature-like instances: 19
- operators: 4
- fare zones: 6
- fare concessions: 3
- subjects with explicit `dcterms:source`: 211
- distinct Wikipedia source URIs represented in the graph: 15

These numbers indicate that the graph is not merely a minimal toy example. It contains enough domain structure to answer fare, accessibility, operator, closure, and journey questions across multiple modes.

Performance metrics

A lightweight performance evaluation was run directly against the committed final graph. On this environment:

- parsing `final_london_transport_kg.ttl` took about 0.80 seconds
- running the full 20-query validation set took about 0.28 seconds in total
- average per-query time was about 0.014 seconds
- peak resident memory during the run was about 49 MB

The slowest query in this run was the King's Cross intersection query, but even that completed in roughly 0.12 seconds. These timings are acceptable for a coursework-sized KG and suggest that the current graph remains easy to load and query on ordinary hardware.

Provenance and source-coverage evaluation

The generated source manifest is a useful secondary metric because it shows how much source material the pipeline was designed to cover:

- 1,299 manifest entries in `docs/generated/tfl-sources.json`
- 695 route-sequence fetches
- 560 stop-point fetches
- 19 line-status fetches

This provides a stronger argument for scalability than the queryable manual ABox alone. However, the namespace split described earlier means that much of this harvested information is not yet visible in the main coursework query namespace.

Suggested baseline comparisons

The coursework brief also asks for comparison against simpler LLM approaches. The repository does not preserve a full baseline experiment, so the most defensible methodology is:

1. Use the 20 competency questions as a fixed evaluation set.
2. Record gold or expected answers from the current validated KG.
3. Ask a plain LLM the same 20 questions without retrieval.
4. Ask the same LLM with KG-backed retrieval.
5. Compare exactness, completeness, hallucination rate, and citation quality.

This comparison is not fully archived in the current repository, so it should be presented as the evaluation method rather than as a completed experimental result.

Validity threats

The evaluation should acknowledge four threats to validity:

1. The automated outputs are currently split across file-based namespaces and are therefore underused by the main SPARQL layer.
2. The repository no longer contains the complete raw source snapshot tree referenced in the manifest.
3. The unstructured extraction stage relies on cached LLM output rather than a fully reproducible live run.
4. Some CQ answers are currently supported by manually seeded ABox facts in the ontology, which improves answerability but reduces the strength of the end-to-end automation claim.

Conclusion

Overall, this repository contains a strong coursework narrative: it defines an ontology for TfL public transport, reuses existing vocabularies appropriately, combines structured and unstructured sources, and validates the resulting KG with 20 SPARQL competency questions that all return answers on the current final artifact.

The main remaining weakness is not the domain framing, but integration quality. The project is closest to a successful submission when described honestly as a hybrid system: a solid manually curated core ontology and CQ layer, supported by an ambitious automated pipeline whose harvested outputs still need namespace normalization and tighter provenance preservation before they can be counted as fully integrated automated population of the same graph.